



Observer Design Pattern in E-commerce Systems

Product Availability Notification System

PRESENTED BY: BYLASAN KHALED AHMAD HAMDAN

The Notification Problem

Scale & Complexity

Modern e-commerce systems handle thousands of concurrent users, making notification management increasingly complex and inefficient.

Out-of-Stock Dilemma

Products frequently go out of stock temporarily, while hundreds of interested users want to be notified the moment they become available again.

Real-World Impact

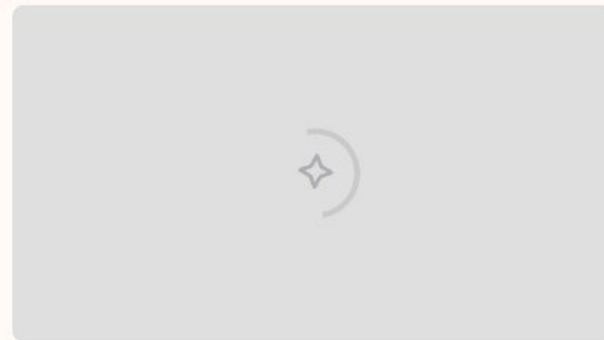
Without a smart notification mechanism, users miss restocks, businesses lose sales, and the overall shopping experience degrades significantly.



Why Traditional Approaches Fail

Conventional object-oriented designs create **tight coupling** between products and users — every user holds a direct reference to the product, making the system brittle and hard to maintain.

The **Observer Design Pattern** breaks this dependency by introducing a subscription model: users register interest in a product, and the system automatically pushes updates whenever the product status changes — no manual polling required.



System Roles at a Glance



The Subject

The **Product** acts as the subject. It holds the availability state and maintains a list of all subscribed observers.



The Observers

Users act as observers. They subscribe to products they care about and wait passively for status updates.



Instant Notification

When availability changes, **all subscribed users** are notified instantly and automatically — no manual checking needed.

Problem Analysis: Traditional vs. Scalable

Manual Polling

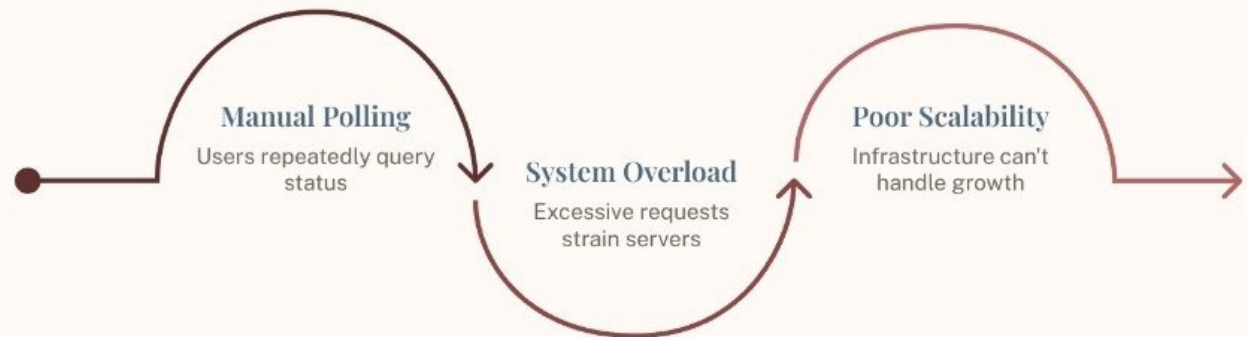
Users must repeatedly query product status, generating unnecessary server load and wasting resources.

Poor User Experience

Constant manual checking is frustrating and leads to missed restocks — users simply give up.

Scalability Wall

As users grow, sending individual updates and managing direct connections becomes exponentially harder to maintain.



Each problem compounds the next — manual polling floods the system, which buckles under scale, forcing costly rewrites.

Why the Observer Pattern?

1

Subscribe

Users register interest in a specific out-of-stock product with a single call.

2

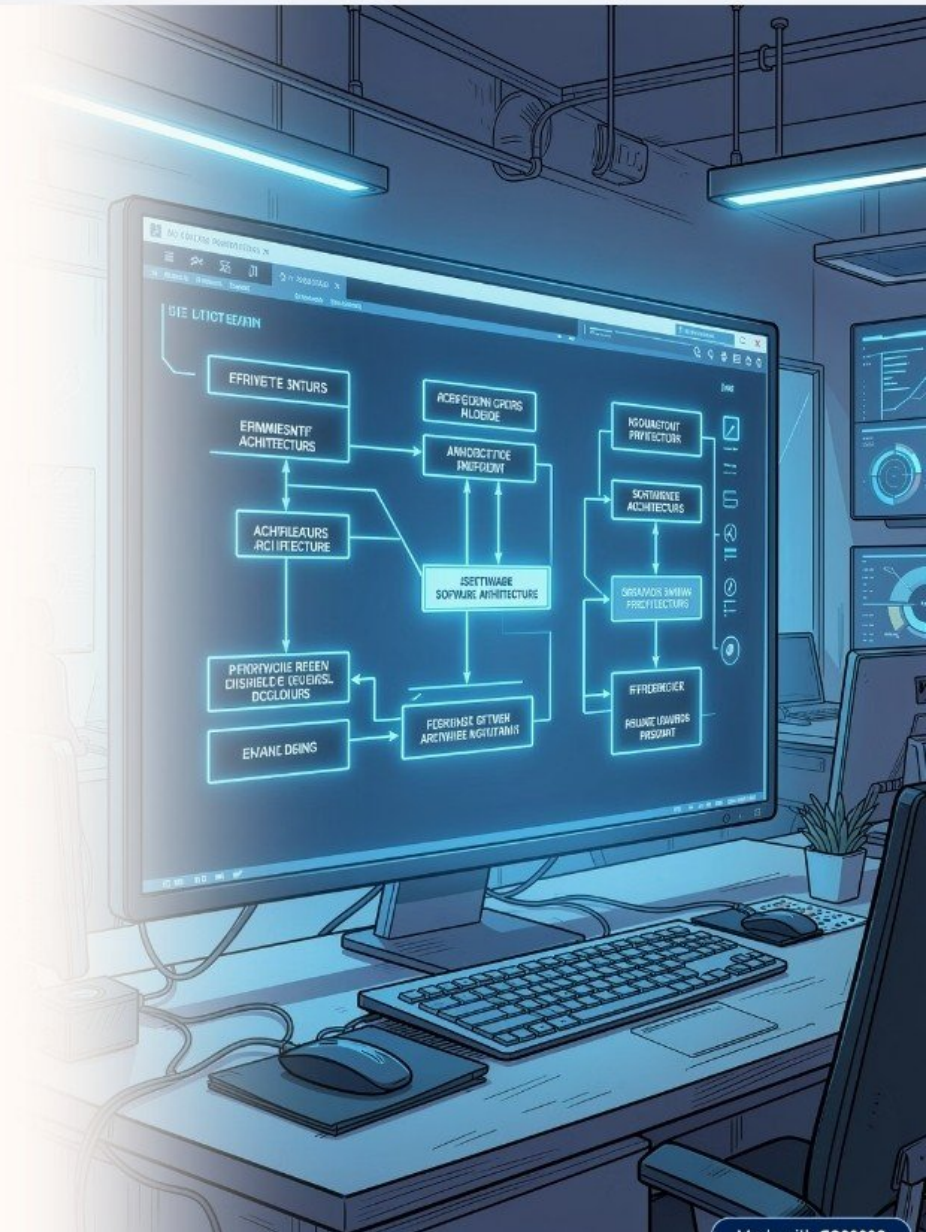
State Changes

Product availability is updated — triggering an automatic internal notification cycle.

3

Notify All

Every subscribed user receives an instant, automatic notification — zero manual effort.



Key Benefits of the Observer Pattern



Loose Coupling

Products and users are independent. Adding or removing subscribers requires no changes to the core product logic.



Zero Manual Polling

Notifications are pushed automatically on state change, eliminating redundant queries and reducing system load.



Highly Scalable

The subscriber list grows dynamically. Whether there are 10 or 10,000 users, the mechanism remains identical and efficient.



Easy to Maintain

The notification logic is centralized inside `notifyObservers()`, so changes propagate from a single location.

Core Interfaces & Implementation

Subject Interface

```
public interface Subject {  
    void subscribe(Observer o);  
    void unsubscribe(Observer o);  
    void notifyObservers();  
}
```

- ⓘ Defines the contract for any observable object. **subscribe()** registers a user, **unsubscribe()** removes them, and **notifyObservers()** broadcasts the update.

Observer Interface

```
public interface Observer {  
    void update(String productName,  
                boolean available);  
}
```

- ⓘ Any class implementing this receives real-time updates whenever a subscribed product changes state.

Product Class

```
public class Product implements Subject {  
    private List<Observer> observers  
        = new ArrayList<>();  
    private boolean available;  
  
    public void subscribe(Observer o) {  
        observers.add(o);  
    }  
    public void unsubscribe(Observer o) {  
        observers.remove(o);  
    }  
    public void notifyObservers() {  
        for (Observer o : observers)  
            o.update(productName, available);  
    }  
    public void setAvailability(  
        boolean available) {  
        this.available = available;  
        notifyObservers();  
    }  
}
```

- ✅ **setAvailability()** is the trigger — any state change automatically fires the full notification cycle.

User Implementation & Live Execution

User Class

```
public class User implements Observer {  
    private final String name;  
  
    @Override  
    public void update(String productName,  
                        boolean available) {  
        if (available) {  
            System.out.println(name  
                + ": " + productName  
                + " is now AVAILABLE!");  
        } else {  
            System.out.println(name  
                + ": " + productName  
                + " is OUT OF STOCK!");  
        }  
    }  
}
```

Main — Execution & Testing

```
Product laptop =  
    new Product("Gaming Laptop");  
User user1 = new User("Ahmad");  
User user2 = new User("Sara");  
  
laptop.subscribe(user1);  
laptop.subscribe(user2);  
  
// Notifies Ahmad & Sara  
laptop.setAvailability(true);  
  
// Sara unsubscribes  
laptop.unsubscribe(user2);  
  
// Notifies Ahmad only  
laptop.setAvailability(false);
```

- ✔ Both Ahmad and Sara receive the first notification. After Sara unsubscribes, only Ahmad is notified — demonstrating dynamic, runtime-controlled subscriptions.

Conclusion

The **Observer Design Pattern** delivers an elegant, production-ready solution for real-time product availability notifications in e-commerce platforms.

Better Organization

Clear separation between subjects and observers keeps the codebase clean and modular.

Reduced Overhead

Eliminates unnecessary polling operations, lowering server load and improving response times.

Flexible & Scalable

Supports a growing user base with zero changes to core logic — simply subscribe and notify.

